Семинар 1. Основы работы с pthreads

1 Pthreads: создание и завершение потоков

`Pthreads (POSIX Threads)` — стандарт POSIX-реализации потоков. Реализации данного API существуют для большого числа UNIX-подобных ОС (GNU/Linux, Solaris, FreeBSD, OpenBSD, NetBSD, OS X).

Для использования в ОС Windows рекомендуется установить `WSL` (`Windows Subsystem for Linux`) — тонкий слой виртуализации, который позволяет запускать приложения `Linux`, не покидая привычного окружения `Windows`. Для установки `WSL` нужно выполнить следующую команду в `PowerShell` (запуск от имени администратора):

```
wsl --install
```

После завершения установки запустите программу `Ubuntu` — это и есть терминал `Linux`.

Логика управления потоками в pthreads в целом соответствует таковой при работе с библиотекой `thread` в `C++`. Отметим, что реализация потоков в `C++` в `Unix`-системах существенно опирается на вызовы функций из `pthreads`. Библиотека `pthreads` предоставляет более низкоуровневый доступ к управлению процессом выполнения потоков, что позволяет добиться более высокой производительности. Поэтому данная библиотека широко используется для создания систем реального времени, высокочастотной торговли `(HFT)` и пр.

Напишем многопоточную версию «Hello, world!».

Листинг 1: Многопоточный «Hello, world»

```
1 #include<stdio.h>
2 #include<stdlib.h>
3 #include<pthread.h>
4
5 int thread_count;
```

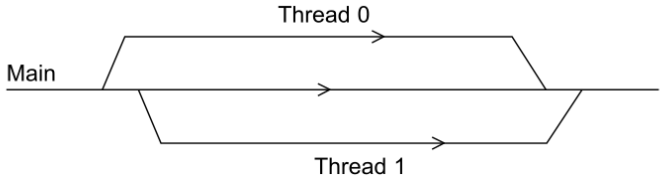


Рис. 1: Создание и завершение потоков

```
 6
 7 void* routine(void* rank){
 8         long my_rank = (long)rank;
 9         printf("Hello from thread %ld of %d\n",
10         my_rank, thread_count);
11         return NULL;
12 }
13
14 int main(int argc, char** argv){
15         thread_count = strtol(argv[1], NULL, 10);
16         pthread_t* thread_handles =
17         malloc(thread_count * sizeof(pthread_t));
18
19         for(long i = 0; i < thread_count; i++){
20                 int err = pthread_create(&thread_handles[i], NULL,
21                                 routine, (void*) i);
22         if(err != 0)
23                 perror("Error");
24 }
25
26         printf("Hello from main thread\n");
27
28         for(int i= 0; i < thread_count; i++)
29                 pthread_join(thread_handles[i], NULL);
30
31         free(thread_handles);
32
33 }
```

Создание потока.

```
pthread_create(
        pthread_t*              thread_p,               /*out*/
        const pthread_attr_t*   attr_p,                 /*in*/
        void*                   (*routine) (void*),     /*in*/
        void*                   args                    /*in*/
);
```

Первый аргумент — это дескриптор потока, куда функция pthread_create запишет данные конкретного создаваемого потока. Гарантируется, что в объекте типа `pthread_t` содержится достаточно информации, чтобы однозначно идентифицировать поток.

Дескрипторы потока можно использовать, например, чтобы понять, соответствуют ли два дескриптора одному или разным потокам

```
int pthread_equal(pthread_t t1, pthread_t t2);
```

Можно получить дескриптор текущего потока

```
pthread_t pthread_self(void);
```

Второй аргумент функции `pthread_create` — атрибуты потока. Эта структура содержит параметры планировщика, размер стека и другие свойства (подробности что и как задать см. тут).

Третий аргумент — это указатель на функцию, запускаемую на потоке.

Четвертый аргумент — параметры, передаваемые в функцию.

Завершение потока

```
int pthread_join(
        pthread_t               thread_p,               /*in*/
        void**                  p__arg                  /*out*/
);

int pthread_detach(
        pthread_t               thread_p,               /*in*/
);

int pthread_kill(
        pthread_t               thread_p,               /*in*/
        int                     sig                     /*in*/
);
```

```
int pthread_cancel(
        pthread_t               thread_p              /*in*/
);

void pthread_exit(
        void**                  retval               /*out*/
);
```

Функция `pthread_join` ждет, когда все потоки завершат свою работу. Далее она помещает в `&p_arg` возвращаемое потоком значение и очищает выделенные потоку ресурсы. Таким образом, когда программы покидает строку 31 программы 1, то мы можем быть уверены в том, что все потоки завершили свою работу, ресурсы, занятые потоками, освобождены, а возвращаемые значения успешно сохранены (если возникла ошибка в процессе, то функция вернет ее код).

Функция `pthread_detach`, напротив, ничего не будет ждать и возвращать из потока. Она просто пометит поток как «отсоединенный» и больше ничего не будет делать. Поток сам завершит свою работу и выделенные на него ресурсы будут возвращены операционной системе. Отличие `pthread_detach` от `pthread_join` можно проиллюстрировать, сравнив результат работы программ 1 и 2 при большом количестве потоков (например, 100000 или больше). Первая программа выдаст ошибку «Cannot allocate memory», т.к. все создаваемые потоки «живут» вплоть до 31 строчки, поэтому память рано или поздно исчерпается. Что же касается второй программы (листинг 2), то каждый вновь созданный поток функцией `detach` тут же «отсоединяется», и по завершению своей работы тут же освобождает ресурсы. Поэтому вторая программа будет работать при любом количестве потоков.

Листинг 2: Отличие join от detach

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>

int thread_count;

void* routine(void* rank){
        long my_rank = (long)rank;
        printf("Hello from thread %ld of %d\n",
```

```
        my_rank, thread_count);
        return NULL;
}

int main(int argc, char** argv){

        thread_count = strtol(argv[1], NULL, 10);

        pthread_t* thread_handles = malloc(thread_count *
                sizeof(pthread_t));

        for(long i =0; i<thread_count; i++){
                int err = pthread_create(&thread_handles[i], NULL,
                routine, (void*) i);
                pthread_detach(thread_handles[i]);
                if(err != 0)
                        perror("Error");
        }

        printf("Hello from main thread\n");

        free(thread_handles);

}
```

Отметим, что вызов `pthread_detach` не эквивалентен «невызову» `pthread_join` в случае, когда мы не хотим ждать завершения работы потока. Если мы просто «забываем» про поток, то он не завершается корректно и превращается в «зомби», таким образом не высвобождая ресурсы.

Функция `pthread_kill` отправляет потоку сигнал, который передается ей в качестве второго аргумента. Константы, соответствующие передаваемым кодам, содержатся в заголовочном файле `signal.h`. Для завершения потока можно передавать константу `SIGQUIT`, при этом поток будет «убит» незамедлительно.

Функция `pthread_exit` завершает текущий поток (тот, из которого она была вызвана), возвращая из него значение `&retval`.

Функция `pthread_cancel` отправляет сигнал завершения потоку, дескриптор которого передается в качестве аргумента. При этом поток мо-

жет как отреагировать на этот сигнал и завершиться (если он отменяемый), либо проигнорировать его (если он неотменяемый). По умолчанию процесс является отменяемым. Чтобы задать отменяемость процесса, используется функция

```
int pthread_setcancelstate(
        int     state,              /*in*/
        int     *oldstate           /*out*/
);
```

Первый аргумент — это константа, значение которой
`PTHREAD_CANCEL_ENABLE` или
`PTHREAD_CANCEL_DISABLE`.
Во втором аргументе можно сохранить текущий статус потока.

Если поток отменяемый, то посылаемый сигнал отмены будет исполнен либо асинхронно, либо отложено (по умолчанию). Асинхронное исполнение означает исполнение в любой момент (как правило, немедленно). Отложенное исполнение означает, что поток завершится, как только встретит одну из команд, являющихся точками отмены. Полный список таких команд см. здесь.

Конкретный тип отмены задается аналогично, с помощью функции

```
int pthread_setcanceltype(
        int     type,               /*in*/
        int     *oldtype            /*out*/
);
```

Первый аргумент — это одна из двух констант
`PTHREAD_CANCEL_DEFERRED`
`PTHREAD_CANCEL_ASYNCHRONOUS`
Во втором аргументе сохраняется текущее значение.

Следующая программа (листинг 3) демонстрирует передачу структуры в качестве аргумента для потока, а также получение возвращаемого потоком значения.

Листинг 3: Передача и возвращение структур из потока

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
```

```c
#include <signal.h>

int thread_count;

struct S{
                long rank;
                char* title;
};

void *routine(void *s){

        struct S* my_s = (struct S*)s;
        printf("Hello from thread %ld of %d, Title: %s\n",
        my_s->rank, thread_count, my_s->title);
        struct S* ret = malloc(sizeof(struct S));
        ret->rank = my_s->rank;
        ret->title = "return";
        return ret;

}

int main(int argc, char **argv){

        thread_count = strtol(argv[1], NULL, 10);

        pthread_t *thread_handles =
                malloc(thread_count * sizeof(pthread_t));

        for (long i = 0; i < thread_count; i++){
                struct S* s = malloc(sizeof(struct S));
                s->rank = i;
                s->title = "Title";
                int err = pthread_create(&thread_handles[i],
                NULL, routine, (void *)s);
                if (err != 0)
                        perror("Error");
        }
```

```
    printf("Hello from main thread\n");

    for (int i = 0; i < thread_count; i++){
        struct S* s;
        pthread_join(thread_handles[i], (void**)&s);
        printf("Return: rank=%ld, title=%s\n", s->rank, s->title
        free(s);
    }

    free(thread_handles);

}
```

2 False sharing

Рассмотрим систему с общей памятью и двумя процессорами.

Пусть $x = 7$ — общая переменная для двух процессоров, a — переменная процессора 1, b, c — переменные процессора 2.

Время	Процессор 1	Процессор 2
0	$a = 2x$	$c = x$
1	$x = 12$	Команды без x
2	—	$b = 10x$

Чему равно b? Если процессор 2 «не знает» об изменении переменной x, сделанном процессором 1, то значение b будет равно 7, в противном случае 120. Обратите внимание, что, даже если кэш-память процессора 1 реализована по модели сквозной записи, то второй процессор все равно может не знать, что ему нужно обратиться к основной памяти за актуальным значением x.

Ситуация, когда процессор «не знает», была ли изменена кэш-строка, с которой он работал, называется некогирентностью кэшей. Применяются следующие способы обеспечения когерентности кэшей: отслеживание и справочник.

При обеспечении когерентности кэшей может возникнуть **false sharing**: ситуация, при которой процессор вынужден заново загружать кэш-строку, даже если та ее часть, с которой работает данное ядро, не менялась.

Явление false sharing проиллюстрируем на примере задачи умножения матрицы на вектор с построчным разбиением (см. листинг 4). Замерим время работы программы с матрицей 8 млн на 8, 8 тыс на 8 тыс и 8 на 8 млн при выполнении на 1, 2 и 4 потоках. Результаты замеров см. в таблице 1.

Листинг 4: Умножение матрицы на вектор

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  #include<pthread.h>
4  #include"timer.h"
5
6  #define N_ITER 100
7
8  int thread_count;
9  int n_rows;
10 int n_cols;
11
12 int* A;
13 int* vec;
14 int* res;
15
16 pthread_t* thread_handles;
17
18 void fill_array(int* arr, int n);
19 void print_vec(int* arr, int n);
20 void print_mat(int* mat, int r, int c);
21 void* routine(void* rank);
22
23 int main(int argc, char** argv){
24
25     thread_count = strtol(argv[1], NULL, 10);
26     n_rows = strtol(argv[2], NULL, 10);
27     n_cols = strtol(argv[3], NULL, 10);
28
29     A = malloc(n_rows*n_cols*sizeof(int));
30     vec = malloc(n_cols*sizeof(int));
31     res = malloc(n_rows*sizeof(int));
```

```
32
33      fill_array(A, n_rows*n_cols);
34      fill_array(vec, n_cols);
35
36      thread_handles = malloc(thread_count*sizeof(pthread_t));
37
38      double start, end;
39      GET_TIME(start);
40      for(int iter=0; iter<N_ITER; ++iter){
41
42      for(long i=0; i<thread_count; ++i)
43          pthread_create(&thread_handles[i], NULL, routine, (void*)i);
44
45      for(int i=0; i<thread_count; ++i)
46          pthread_join(thread_handles[i], NULL);
47
48      }
49      GET_TIME(end);
50      printf("%.3lf\n",(end-start)/N_ITER);
51
52      free(A);
53      free(vec);
54      free(res);
55      free(thread_handles);
56
57  }
58
59
60  void fill_array(int* arr, int n){
61      for(int i=0; i<n; ++i)
62          arr[i] = rand()%5;
63  }
64
65  void* routine(void* rank){
66      long my_rank = (long) rank;
67      int my_rows = n_rows / thread_count;
68      int my_first_i = my_rows*my_rank;
69      int my_last_i = (my_rank+1)*my_rows-1;
```

Кол-во потоков	8 млн x 8	8 тыс x 8 тыс	8 x 8 млн
1	0.124	0.098	0.1
2	0.063	0.05	0.096
4	0.032	0.026	0.118

Таблица 1: Результаты замеров времени работы программы из листинга 4

```
70      for(int i=my_first_i; i<=my_last_i; ++i){
71          res[i] = 0;
72          for(int j=0; j<n_cols; ++j)
73              res[i] += A[i*n_cols+j]*vec[j];
74      }
75      return NULL;
76  }
```

Обращают на себя внимание следующие два принципиальных момента.

Во-первых, время работы программы на одном потоке отличается в зависимости от входных данных. Различие связано с кэш-промахами чтения и записи. Промахи записи возникают в строке 71 при попытке обнулить элемент вектора. Промахи чтения возникают в строке 73 при доступе к компонентам вектора-столбца, на который умножается матрица.

Во-вторых, в то время как квадратная матрица и высоко-худощавая матрица позволяют получить близкое к линейному ускорению программы, то толсто-короткая матрица ведет себя намного хуже. На ряде архитектур может наблюдаться даже замедление программы.

Поясним, почему при умножении матрицы 8 на 8 млн на вектор 8 млн ускорение оказывается существенно меньшим, чем в двух других случаях. Это связано с рассмотренным выше false sharing'ом. Типичный размер кэш-строки составляет 64 байта, т.е. 8 чисел типа `double`. Результирующая строка y в рассматриваемом случае состоит из 8 чисел типа double, то есть из одной кэш-строки. В различные элементы этого массива разные процессоры пишут свою результаты вычислений. При этом не возникает состояния гонки, ибо каждый процессор работает со своим набором индексов. Наборы индексов разных ядер не пересекаются. Однако, в силу того, что весь массив относится к одной строке кэша, при любом

изменении этого массива, каждый процессор считает, что хранящиеся у него значения y более не актуальны и нужно их обновить, обратившись к оперативной памяти. Из-за этого и возникает задержка, объясняющая снижение эффективности.

Отметим, что на самом деле никакой необходимости обновлять данные у процессора нет, ибо каждый процессор изменяет только свою часть массива, не затрагивая область ответственности другого ядра. Отсюда слово false.

Почему описанная проблема не возникает в двум других случаях входных данных? Потому что массив y больше и не умещается в одну строку кэша. Например, если длина y 8 тысяч элементов, то это тысяча строк кэша. Крайне маловероятно, что второй поток будет обрабатывать начало своей части массива в то время как первый поток дошел уже до конца своей части. Следовательно, явление false sharing, скорее всего, не возникнет.

Как решить эту проблему с рассматриваемом случае? Заполнить массив y «мусором» так, чтобы он более не умещался в одну строку.